Graduate Team 2: Preventive and Perfective Maintenance in WeeWX

CS 5340 Software Maintenance:
Semester: Spring
Final Presentation

Team members:
Andrew Nguyen
Jerry Delva

Permission is granted for publication on the UCCS website.

Professor: Dr. Armin Moin
Teaching Assistant (TA): Sahana Vinayak

Project Overview (Jerry)

WeeWX represents a **long-lived, evolving open-source system**

Multiple contributors increase **maintenance complexity over time**

Real-world usage makes maintainability issues **high-impact**

Continuous feature additions require:

- Ongoing refactoring
- Preventive maintenance to avoid technical debt

Ideal subject for studying **maintenance risk in legacy systems**

Why this helps:

This explicitly frames WeeWX as a **maintenance case study**

Relationship to Course Content (Jerry)

Legacy Systems & Maintenance Challenges

- WeeWX exhibits characteristics of a **software aging system**
- Incremental changes increase entropy without active maintenance

Software Evolution & Reengineering/Refactoring

- Demonstrates Lehman's laws
- Increasing complexity without refactoring / Need for continuous adaptation

Maintenance Types

- Preventive maintenance justified by **metric trends**
- Perfective maintenance justified by **measured code quality decline**

Maintenance Approach: Preventive and Perfective (Jerry)

Preventive:

- Detect rising complexity before defects emerge / Reduce technical debt accumulation
- Maintain stable architecture during feature growth

Perfective:

- Refactoring improves:Modularity / Understandability / Long-term adaptability
- Cleaner code improves contributor productivity

Why both?:

Preventive = **risk detection**

Perfective = **risk reduction**

Metrics link both into a repeatable maintenance strategy

Customers,End Users & Values (Andrew)

End Users:

- Meteorologists / Weather hobbyists

Customers:

- Open-source contributors
- WeeWX maintainers

Values:

- Enables **data-driven maintenance planning**
- Helps maintainers prioritize **high-risk code areas**
- Reduces long-term cost of change through early detection
- Improves onboarding for new contributors via cleaner code
- Encourages sustainable open-source development practices
- Supports **objective discussions** about refactoring needs

Research Question & Hypothesis (Andrew)

Research Question:

- What measurable maintenance risks exist in WeeWX codebase ?
- Can static code metrics detect maintainability degradation across versions?

Hypothesis:

Modules with rising complexity will show:

- Declining maintainability
- Increased maintenance effort

Static metrics can guide **preventive maintenance decisions**

Focuses on **structural quality**, not functional correctness

Emphasizes **trend analysis across versions**

Looks for early warning signs before defects appear

Differentiator/Unique Selling Point (Andrew)

- Applies **longitudinal static analysis** across multiple WeeWX versions
- Combines **preventive + perfective maintenance** in a single metric-driven framework
- Focuses on **open-source, real-world legacy code**, not synthetic examples
- Identifies **maintenance risk trends over time**, not just snapshot metrics
- Produces **actionable refactoring targets** for maintainers
- Bridges **academic metrics** with **practical maintenance decisions**

Related Work (Jerry)

Static Metrics & Maintainability

- WeeWX official documentation provides architectural context and evolution history
- Coleman et al. (1994) introduce the **Maintainability Index**, combining:
  - Cyclomatic Complexity / Lines of Code / Halstead metrics
- Gill & Kemerer (1991) empirically link **Cyclomatic Complexity** to:
  - Maintenance effort / Maintenance productivity
- Prior work validates CC and MI as reliable indicators of:
  - Maintenance difficulty / Long-term system quality

Relevance to Our Study

- Supports using **static metrics** to identify maintenance risk
- Justifies focusing on **complexity trends over time**, not single snapshots

Related Work (Jerry)

Static Analysis & Metric Trend Studies

- Vogelsang et al. (2010) show that:
  - Static metrics can be integrated directly into analysis tools
  - Continuous metric tracking improves maintainability monitoring

- Császár & Cserép (2026) demonstrate:
  - CC, MI, and cohesion metrics reveal code smells in open-source systems
  - Metric-based analysis highlights problematic modules early

- Karanikiotis et al. (2021) find:
  - **Metric trends over time** outperform single-version analysis
  - Early detection enables preventive maintenance interventions
  - Longitudinal static analysis enables detection of maintainability trends across software version

Connection to WeeWX

- Confirms value of **longitudinal analysis**
- Reinforces static metrics as effective tools for **preventive maintenance**

Methodology,Metric Intrepretation, and Experimental Setup(Andrew)

Methodology

- Selected **weecfg/update_config.py** from the two comparable WeeWX releases: v5.2.0 and v5.3.1
- Compared the same file across versions to identify persistent maintainability risk over time:
- Evaluated:
  - Cyclomatic Complexity(CC) at function level
  - Maintainaiblity Index(MI) at file level
  - Pylint warnings related to maintainability and design
- Used the version comparison to determine whether update_config.py contained stable refactoring targets

Metric Interpretation

- CC scoring (Radon): A = 1–5, B = 6–10, C = 11–20, D = 21–30, E = 31–40, F = 41+
- MI scoring(Radon): A = 20–100, B = 10–19, C = 0–9
- In this study:
  - Higher CC = higher function-level maintenance risk
  - Lower MI - lower file-level maintainability

Experimental Setup

- Baseline comparison: v5.2.0 vs. v5.3.1
- Follow-up evaluation: original v5.3.1 vs refactored update_config.py

Cross-Version Findings and Refactoring Targets(Andrew)

Cross-Version Findings for update_config.py

- update_config.py remained a stable maintainability hotspot across both versions
- File-level MI stayed low:
  - v5.2.0: 26.43
  - v5.3.1: 26.43
- The highest-complexity functions were unchanged across both releases:
  - update_to_v26: CC 39 (E)
  - update_to_v25: CC 25 (D)
- Pylint on v5.3.1 reinforced the same maintenace concerns:
  - too many branches
  - too many statements
  - possible used-before-assignment

Refactoring Targets

- Based on the version comparison, we selected: **update_to_v26** and **update_to_v25** functions
- These functions were chosen because they showed the strongest and most persistent complexity signal in update_config.py

Experimental Results:Refactoring impact on update_config.py (Andrew)

Post Refactoring Results

- After refactoring both selected functions showed substantial CC reduction:
  - update_to_v26: 39 (E) → 2 (A)
  - update_to_v25: 25 (D) → 3 (A)
- File-level MI improved:
  - 26.43 → 30.73

Interpretation

- Cross-version analysis identified a persistent maintenance hotspot
- Targeted refactoring produced measurable structural improvement
- The results support using static metrics to guide both:
  - Pefective maintenance through hot spot identification
  - Perfective maintenance through refactoring evalaution

Live Demo

Conclusion and Future Work(Jerry)

Conclusion

- Static code metrics effectively expose **hidden maintenance risks**
- Preventive maintenance can reduce long-term complexity growth
- Perfective maintenance improves **readability, structure, and sustainability**
- WeeWX benefits from **continuous metric monitoring**

Future Work

- Extend analysis to:
  - **C**oupling **B**etween **O**bjects & **L**ack of **C**ohesion **M**ethods (LCOM, CBO)
    LCOM measures how well a class's methods belong together, while CBO measures how strongly a class is coupled to others—together they help identify design and maintainability risks in evolving software systems.
  - Change frequency and defect history

- Automate metric tracking using **CI/CD integration**
- Apply framework to **other open-source legacy systems**
- Validate findings with **maintainer feedback**

Project Task completed(Jerry)

Project Setup & Data Collection (Andrew & Jerry)

- Set up WeeWX environment locally / Selected comparable **older and current WeeWX versions**
- Identified common modules for longitudinal analysis

Static Analysis Execution (Andrew)

- Ran **Radon** to compute:
  - Cyclomatic Complexity / Maintainability Index

- Ran **Pylint** to detect:
  - Code smells / Style and design issues

Analysis & Interpretation (Andrew)

- Collected and organized metric outputs / Identified **maintenance hotspots** and high-risk modules
- Analyzed **complexity and maintainability trends** across versions

Maintenance Evaluation (Andrew & Jerry)

- Translated metric results into **maintainability risks**
- Proposed:
  - **Preventive maintenance** (early risk detection)
  - **Perfective maintenance** (refactoring and readability improvements)

Documentation & Presentation (Jerry)

- Compiled related academic research supporting metric-driven maintenance
- Created tables/visual summaries of analysis results
- Coordinated final report and presentation to align with course requirements

Open Source, License, and Repository

Project Repository:
https://github.com/anguyen7-99/CS-5340-Group-2.git

GitHub Projects link:
https://github.com/users/anguyen7-99/projects/1

WeeWx Repository:
https://github.com/weewx/weewx

License:
GNU Public License v3

University of Colorado
Colorado Springs

University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus